# visualize_schedule_overhead

February 12, 2026

## 1 Figure 16. Results in large-scale simulations (b) Scheduling Latency

## 2 Visualize Schedule Overhead

Compare the scheduling overhead for each job.

### 2.0.1 Define Generic Helper Function

[1]:
```
from Suika import *
import os
import torch
import numpy as np
import matplotlib.pyplot as plt

result_path = "../results/schedule-overhead/{policy}-{n_gpu}.pth"

show_fig = True

def load(policys, n_gpu):
  data = {}
  for policy in policys:
    try:
      data[policy] = torch.load(result_path.format(policy=policy, n_gpu=n_gpu),␣
 ↪weights_only=False)
      print(f"{policy}-{n_gpu} loaded with {len(data[policy])} entry")

    except FileNotFoundError:
      print(f"File not found for {policy}-{n_gpu}")
  return data

def compare_percentile(data, percentiles=[50, 90, 95, 99]):
  # for each percentile, get the value for each policy, rank, and print raw␣
 ↪value with relative ratio
  result = {}
  for p in percentiles:
    print(f"--- Percentile {p} ---")
```

```
    vals = []
    for policy, values in data.items():
      if len(values) == 0:
        continue
      val = np.percentile(values, p)
      vals.append((policy, val))
    vals.sort(key=lambda x: x[1])
    if len(vals) == 0:
      print("No data")
      continue
    base_val = vals[0][1]
    for policy, val in vals:
      ratio = val / base_val
      print(f"  {policy}: {val:.2f} ({ratio:.2f}x)")
    print()
    result[p] = vals
  return result # key: percentile, value: list of (vals[0], vals[1], ...) (each␣
↪mapped to a policy)
```

```
/home/fake/.local/lib/python3.10/site-packages/tqdm/auto.py:21: TqdmWarning:
IProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

### 2.0.2 Show CDFs for manual inspection

[2]:
```
n_gpus = [64, 128, 256, 512, 1024]
policys = ["suika", "sia", "rubick"]
policys_map = {
  "suika": "Suika",
  "sia": "Sia",
  "rubick": "Rubick"
}

precentiles_result = {}
for n_gpu in n_gpus:
  print(f"=== {n_gpu} GPUs ===")
  data = load(policys, n_gpu)
  precentiles_result[n_gpu] = compare_percentile(data)
```

```
=== 64 GPUs ===
suika-64 loaded with 7647 entry
sia-64 loaded with 7614 entry
rubick-64 loaded with 7310 entry
--- Percentile 50 ---
  rubick: 0.00 (1.00x)
```

```
  sia: 0.00 (35.37x)
  suika: 0.02 (147.13x)

--- Percentile 90 ---
  rubick: 0.00 (1.00x)
  sia: 0.01 (4.55x)
  suika: 0.07 (51.15x)

--- Percentile 95 ---
  rubick: 0.00 (1.00x)
  sia: 0.01 (5.33x)
  suika: 0.08 (52.52x)

--- Percentile 99 ---
  rubick: 0.00 (1.00x)
  sia: 0.01 (6.17x)
  suika: 0.11 (71.29x)

=== 128 GPUs ===
suika-128 loaded with 3116 entry
sia-128 loaded with 2939 entry
rubick-128 loaded with 2896 entry
--- Percentile 50 ---
  rubick: 0.00 (1.00x)
  sia: 0.00 (27.56x)
  suika: 0.03 (183.75x)

--- Percentile 90 ---
  rubick: 0.00 (1.00x)
  sia: 0.01 (2.35x)
  suika: 0.17 (51.34x)

--- Percentile 95 ---
  rubick: 0.00 (1.00x)
  sia: 0.01 (2.18x)
  suika: 0.43 (110.40x)

--- Percentile 99 ---
  rubick: 0.00 (1.00x)
  sia: 0.01 (2.79x)
  suika: 0.54 (119.81x)

=== 256 GPUs ===
suika-256 loaded with 4017 entry
sia-256 loaded with 3766 entry
rubick-256 loaded with 3804 entry
--- Percentile 50 ---
  rubick: 0.00 (1.00x)
```

```
  sia: 0.01 (15.88x)
  suika: 0.09 (218.21x)

--- Percentile 90 ---
  rubick: 0.01 (1.00x)
  sia: 0.01 (1.26x)
  suika: 0.26 (26.91x)

--- Percentile 95 ---
  rubick: 0.01 (1.00x)
  sia: 0.01 (1.05x)
  suika: 0.50 (35.47x)

--- Percentile 99 ---
  rubick: 0.02 (1.00x)
  sia: 0.02 (1.18x)
  suika: 0.81 (39.02x)

=== 512 GPUs ===
suika-512 loaded with 4124 entry
sia-512 loaded with 3945 entry
rubick-512 loaded with 3941 entry
--- Percentile 50 ---
  rubick: 0.00 (1.00x)
  sia: 0.01 (6.44x)
  suika: 0.32 (237.88x)

--- Percentile 90 ---
  sia: 0.02 (1.00x)
  rubick: 0.04 (1.55x)
  suika: 2.33 (98.33x)

--- Percentile 95 ---
  sia: 0.03 (1.00x)
  rubick: 0.05 (1.48x)
  suika: 4.00 (123.47x)

--- Percentile 99 ---
  rubick: 0.08 (1.00x)
  sia: 0.13 (1.60x)
  suika: 8.45 (103.06x)

=== 1024 GPUs ===
suika-1024 loaded with 4972 entry
sia-1024 loaded with 4268 entry
rubick-1024 loaded with 4593 entry
--- Percentile 50 ---
  rubick: 0.00 (1.00x)
```

```
  sia: 0.02 (13.59x)
  suika: 0.54 (422.16x)

--- Percentile 90 ---
  sia: 0.12 (1.00x)
  rubick: 0.20 (1.66x)
  suika: 4.01 (32.95x)

--- Percentile 95 ---
  sia: 0.29 (1.00x)
  rubick: 0.29 (1.00x)
  suika: 5.79 (20.25x)

--- Percentile 99 ---
  rubick: 0.51 (1.00x)
  sia: 8.78 (17.23x)
  suika: 21.58 (42.37x)
```

## 3 Plot P{N} Latency with Cluster Size

[3]:
```python
from matplotlib.ticker import FuncFormatter, FixedLocator, NullLocator

def plot_latency_with_errorbars(precentiles_result, n_gpus, policys_map,␣
↪policies_to_plot,
                                errbar_policies,
                                central_percentile=90, lower_percentile=50,␣
↪upper_percentile=99):
    """
    Plot latency vs. cluster size using a central percentile as the line (e.g.,␣
↪P90)
    and lower/upper percentiles as asymmetric error bars (e.g., P50/P99).

    Args:
        precentiles_result (dict): {n_gpu: {percentile: [(policy, value), ...]}}
        n_gpus (list[int]): x-axis values.
        policys_map (dict): mapping from policy key to display name.
        policies_to_plot (list[str]): policies to include.
        central_percentile (int): percentile used for the line.
        lower_percentile (int): lower error bar percentile.
        upper_percentile (int): upper error bar percentile.
    """
    fig, ax = plt.subplots(figsize=(6, 4))

    marker_map = {
        'suika': 'o',
```

```
        'rubick': '^',
        'sia': 's'
    }

    def get_val(entries, policy):
        return next((v for pol, v in entries if pol == policy), np.nan)

    texts = []
    for policy in policies_to_plot:
        y, err_lo, err_up = [], [], []
        for n in n_gpus:
            percs = precentiles_result.get(n, {})
            c = get_val(percs.get(central_percentile, []), policy)
            lo = get_val(percs.get(lower_percentile, []), policy)
            up = get_val(percs.get(upper_percentile, []), policy)

            y.append(c)
            if np.isnan(c) or np.isnan(lo) or np.isnan(up):
                err_lo.append(np.nan)
                err_up.append(np.nan)
            else:
                err_lo.append(max(c - lo, 0))
                err_up.append(max(up - c, 0))

        y = np.asarray(y, dtype=float)
        yerr = np.asarray([err_lo, err_up], dtype=float)

        marker = marker_map.get(policy)

        if policy in errbar_policies:
            ax.errorbar(
                n_gpus,
                y,
                yerr=yerr,
                fmt='-',
                marker=marker,
                markersize=7,
                capsize=4,
                linewidth=1.5,
                label=f"{policys_map.get(policy, policy)} (err:␣
↪P{lower_percentile}/P{upper_percentile})"
            )
        else:
            ax.plot(
                n_gpus,
                y,
                '-',
```

```
            marker=marker,
            markersize=7,
            linewidth=1.5,
            label=f"{policys_map.get(policy, policy)} "
        )

    # # Optional data labels
    # for i, val in enumerate(y):
    #     if not np.isnan(val):
    #         texts.append(ax.text(n_gpus[i], val, f"{val:.2f}"))

# Axes formatting
ax.set_xlabel("Cluster Size", fontsize=16, fontweight='bold')
ax.set_ylabel("P90 Schedule Latency", fontsize=16, fontweight='bold')

# Log scales
ax.set_xscale('log', base=2)
ax.set_yscale('log', base=10)

# X ticks
ax.set_xticks(n_gpus)
ax.get_xaxis().set_major_formatter(plt.ScalarFormatter())

# Y ticks formatting
def time_formatter(y, pos):
    if y < 1:
        return f"{int(y*1000)}ms"
    else:
        return f"{int(y)}s"
def minor_time_formatter(y, pos):
    if y >= 10:
        return f"{int(y)}s"
    else:
        return ""
ax.yaxis.set_major_formatter(FuncFormatter(time_formatter))
ax.yaxis.set_minor_formatter(FuncFormatter(minor_time_formatter))

# ---- FIXED MINOR TICKS ----
# make sure limits are up to date
ax.set_ylim(0.001, 39)
ax.relim()
ax.autoscale_view()
ymin, ymax = ax.get_ylim()

if ymax <= 10:
    ax.yaxis.set_minor_locator(NullLocator())
else:
```

```
        subs = np.arange(2, 10)
        min_exp = max(1, int(np.floor(np.log10(ymin))))  # start at 10^1
        max_exp = int(np.floor(np.log10(ymax)))
        minor_ticks = []
        for e in range(min_exp, max_exp + 1):
            minor_ticks.extend((subs * (10 ** e)).tolist())
        minor_ticks = [t for t in minor_ticks if 10 <= t <= ymax]
        minor_ticks += [10 ** e for e in range(min_exp, max_exp + 1) if 10 ** e␣
↪<= ymax]
        if minor_ticks:
            ax.yaxis.set_minor_locator(FixedLocator(minor_ticks))
        else:
            ax.yaxis.set_minor_locator(NullLocator())

    ax.tick_params(axis='x', labelsize=15)
    ax.tick_params(axis='y', labelsize=14)

    # Grids
    ax.grid(True, which='major', axis='y', linestyle='-', linewidth=0.6)
    ax.grid(True, which='minor', axis='y', linestyle=(0,(1, 4)), linewidth=0.8)
    ax.grid(True, which='major', axis='x', linestyle='-', linewidth=0.6)


    handles, labels = ax.get_legend_handles_labels()

    order = ['suika', 'sia', 'rubick']

    # Create a mapping from policy name to the handle and label
    legend_map = {}
    for handle, label in zip(handles, labels):
        for policy_key, policy_name in policys_map.items():
            if label.startswith(policy_name):
                legend_map[policy_key] = (handle, label)
                break

    # Get the handles and labels in the desired order
    ordered_handles = [legend_map[key][0] for key in order if key in legend_map]
    ordered_labels = [legend_map[key][1] for key in order if key in legend_map]

    ax.legend(ordered_handles, ordered_labels, fontsize=12)
    plt.tight_layout()

    fig_dir = "../fig"
    os.makedirs(fig_dir, exist_ok=True)
    plt.savefig(f"{fig_dir}/schedule_latency.pdf")
    plt.show()
```

```
# Example usage: P90 as line with P50/P99 error bars
policies_to_plot = ['suika', 'sia', 'rubick']
plot_latency_with_errorbars(
    precentiles_result,
    n_gpus,
    policys_map,
    policies_to_plot,
    ['suika'],
    central_percentile=90,
    lower_percentile=50,
    upper_percentile=99,
)
```

